

# RABBITHOLE MAPPER

Analisis y Mapeo de Algoritmos de Recomendacion  
mediante Agentes Autonomos con Reinforcement Learning y Neo4j

Documento de Diseno y Arquitectura — v1.0

|                     |                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Titulo completo:    | Mapeo Topologico de Algoritmos de Recomendacion en Formatos Short-Video mediante Agentes Autonomos LLM y Bases de Datos de Grafos |
| Tipo de proyecto:   | Trabajo de Fin de Grado (TFG)                                                                                                     |
| Lenguaje principal: | Python 3.11+                                                                                                                      |
| Base de datos:      | Neo4j (Grafos)                                                                                                                    |
| Modelo LLM:         | Llama 3.1 8B (via Ollama, local)                                                                                                  |
| Hardware objetivo:  | NVIDIA RTX 3070 (8 GB VRAM)                                                                                                       |
| Duracion estimada:  | 7 semanas                                                                                                                         |

# Indice de contenidos

---

1. Resumen ejecutivo
2. Motivacion y problema de investigacion
3. Objetivos del proyecto
4. Marco teorico
  - 4.1 Algoritmos de recomendacion como caja negra
  - 4.2 Camaras de eco y polarizacion
  - 4.3 Agentes autonomos con LLM
  - 4.4 Reinforcement Learning aplicado
  - 4.5 Bases de datos de grafos (Neo4j)
5. Arquitectura del sistema
  - 5.1 Vision general (4 capas)
  - 5.2 Capa 1 — Extraccion de datos (Playwright)
  - 5.3 Capa 2 — Agentes LLM (Ollama + Llama 3.1 8B)
  - 5.4 Capa 3 — Reinforcement Learning (Q-Learning)
  - 5.5 Capa 4 — Big Data y analisis de grafos (Neo4j)
6. Perfiles de agente
7. Funcion de recompensa (reward function)
8. Modelo de datos en Neo4j
9. Stack tecnologico completo
10. Plan de desarrollo (7 semanas)
11. Minimo producto viable (MVP)
12. Resultados esperados
13. Limitaciones y trabajo futuro
14. Justificacion academica por asignatura
15. Glosario tecnico

# 1. Resumen ejecutivo

---

## Descripción del proyecto

RabbitHole Mapper es un sistema multi-agente que simula el comportamiento de distintos perfiles de usuario en plataformas de video corto (YouTube Shorts), con el objetivo de mapear y cuantificar como los algoritmos de recomendación generan cámaras de eco y polarización de contenido. Los agentes aprenden y refinan su comportamiento mediante Reinforcement Learning (Q-Learning), y toda la estructura de recomendaciones se almacena y analiza en una base de datos orientada a grafos (Neo4j), permitiendo aplicar algoritmos de centralidad y camino mínimo para obtener conclusiones empíricas y reproducibles.

El proyecto cubre de forma natural las cinco asignaturas del ciclo: Modelos de IA, Programación de IA, Sistemas de Aprendizaje Autónomos, Sistemas Big Data y Big Data Aplicado. Está diseñado para ser completado en 7 semanas con un presupuesto de infraestructura cero, ejecutándose completamente en local sobre hardware con GPU NVIDIA RTX 3070.

## 2. Motivacion y problema de investigacion

---

Los algoritmos de recomendacion de plataformas como TikTok o YouTube son sistemas opacos. Las propias companias no publican su funcionamiento interno y los estudios academicos existentes dependen de encuestas subjetivas o de datos cedidos por las mismas plataformas, lo que introduce un sesgo estructural en la investigacion.

Todo el mundo afirma intuitivamente que estos algoritmos crean 'burbujas' y 'madrigueras de conejo' (rabbit holes). Sin embargo, nadie ha podido demostrar esto de forma independiente, automatizada y a escala, porque hacerlo manualmente requeriria decenas de investigadores viendo videos durante semanas y anotando cada recomendacion.

### Propuesta de solucion

RabbitHole Mapper resuelve exactamente este problema: sustituye a los investigadores humanos por agentes de IA que navegan de forma autonoma, piensan como perfiles demograficos reales y generan un dataset empirico y reproducible que puede auditarse independientemente.

### Preguntas de investigacion que responde este TFG:

- PQ1: A cuantas recomendaciones de distancia se encuentra el contenido extremo desde un video neutro segun cada perfil de usuario?
- PQ2: Con que velocidad queda atrapado cada perfil en su camara de eco?
- PQ3: Existen 'nodos puente' que actuan como puerta de entrada a nichos toxicos para todos los perfiles?
- PQ4: Los grafos de distintos perfiles son topologicamente disjuntos (ausencia de nodos compartidos)?

### 3. Objetivos del proyecto

---

#### Objetivo general

Desarrollar una metodología reproducible para auditar de forma independiente los algoritmos de recomendación de plataformas de video corto, cuantificando mediante análisis de grafos el grado de polarización y aislamiento que generan sobre distintos perfiles de usuario.

#### Objetivos específicos

- OE1: Implementar un sistema de extracción automatizada de metadatos de video (transcripciones, hashtags, títulos) mediante navegación controlada por Playwright.
- OE2: Diseñar e implementar agentes autónomos basados en LLM con perfiles psicográficos diferenciados capaces de tomar decisiones de navegación coherentes.
- OE3: Implementar un mecanismo de Reinforcement Learning (Q-Learning) que permita a los agentes aprender y refinar su comportamiento de navegación a lo largo del tiempo.
- OE4: Modelar la estructura de recomendaciones como un grafo dirigido en Neo4j con nodos (videos) y aristas (relaciones de recomendación).
- OE5: Aplicar algoritmos de análisis de grafos (PageRank, Shortest Path, detección de comunidades) para extraer conclusiones cuantitativas sobre polarización.
- OE6: Generar visualizaciones interactivas del grafo que permitan demostrar visualmente la formación de cámaras de eco.

## 4. Marco teorico

---

### 4.1 Algoritmos de recomendacion como caja negra

Los sistemas de recomendacion modernos utilizan filtrado colaborativo, modelos de aprendizaje profundo y senales implicitas de comportamiento (tiempo de reproduccion, interacciones, historico) para predecir el siguiente contenido. Al no publicarse su arquitectura interna, se les denomina academicamente 'cajas negras'. La unica forma de estudiarlos es mediante auditoria de comportamiento: observar sus inputs y outputs sin acceder al interior.

### 4.2 Camaras de eco y polarizacion de contenido

Una camara de eco es un entorno informativo en el que el usuario unicamente recibe contenido que refuerza sus creencias e intereses previos, sin exposicion a perspectivas alternativas. La polarizacion ocurre cuando el sistema de recomendacion prioriza el contenido emocionalmente intenso (que maximiza la retencion y el engagement) sobre el contenido neutro, empujando progresivamente al usuario hacia contenido mas extremo dentro de su nicho.

### 4.3 Agentes autonomos con LLM

Un agente autonomo basado en LLM es un modelo de lenguaje que, en lugar de responder a preguntas de forma pasiva, toma decisiones y ejecuta acciones sobre un entorno. En este proyecto, el entorno es el feed de YouTube Shorts y la accion disponible es consumir o rechazar cada video. La personalidad del agente se define mediante System Prompting, una tecnica que inyecta un contexto de rol antes de cada inferencia sin modificar los pesos del modelo.

### 4.4 Reinforcement Learning aplicado

El Reinforcement Learning (RL) es un paradigma de aprendizaje en el que un agente aprende a tomar decisiones en un entorno mediante prueba y error, maximizando una senal de recompensa acumulada. Q-Learning es un algoritmo de RL basado en tablas que estima el valor esperado de cada par (estado, accion) sin necesidad de conocer el modelo del entorno, siendo ideal para entornos discretos como el nuestro.

### 4.5 Bases de datos de grafos — Neo4j

Neo4j es un sistema gestor de bases de datos orientado a grafos que almacena la informacion como nodos (entidades) y aristas (relaciones). A diferencia de las bases de datos relacionales, permite ejecutar consultas de caminos y centralidad de forma nativa y eficiente. El lenguaje de consulta propio de Neo4j es Cypher, disenado especificamente para recorrer y analizar grafos.

## 5. Arquitectura del sistema

### 5.1 Vision general — 4 capas

| Capa | Nombre                 | Tecnologia                          | Responsabilidad                               |
|------|------------------------|-------------------------------------|-----------------------------------------------|
| 1    | Extraccion de datos    | Playwright + YouTube Transcript API | Navegar, capturar video y extraer metadatos   |
| 2    | Agentes LLM            | Ollama + Llama 3.1 8B               | Evaluar contenido y generar decision JSON     |
| 3    | Reinforcement Learning | Q-Learning (numpy)                  | Aprender que acciones maximizan la recompensa |
| 4    | Big Data y Grafos      | Neo4j + neovis.js                   | Almacenar, analizar y visualizar el grafo     |

#### Flujo de datos entre capas:

```
[YouTube Shorts] → Playwright captura metadatos → LLM Agent evalua contenido → Decision JSON → Q-Learning actualiza Q-table → Accion ejecutada en navegador → Siguiete video recomendado → Neo4j registra arista → Analisis de grafo → Visualizacion 3D
```

### 5.2 Capa 1 — Extraccion de datos

Playwright controla un navegador Chromium sin interfaz grafica (headless). Al cargar cada video de YouTube Shorts extrae: titulo del video, ID de YouTube, transcripcion completa del audio mediante youtube-transcript-api, hashtags y categoria declarada, y el ID del siguiente video recomendado.

Para evitar deteccion como bot se implementan: delays aleatorios entre acciones (2-8 segundos), movimientos de cursor simulados, y rotacion de User-Agent.

### 5.3 Capa 2 — Agentes LLM

Cada agente es una instancia de Llama 3.1 8B ejecutada localmente mediante Ollama. Recibe el system prompt con su perfil y el contenido del video actual, y devuelve obligatoriamente un JSON estructurado con su decision:

```
{ "categoria_detectada": "gaming", "afinidad_con_perfil": 0.85, "accion": "ver_completo", "like": true, "razonamiento": "Contenido de gaming competitivo, encaja con perfil gamer" }
```

### 5.4 Capa 3 — Reinforcement Learning (Q-Learning)

El agente aprende que acciones de navegacion producen mejores recomendaciones para su perfil. El entorno se define formalmente como un MDP (Markov Decision Process):

| Componente MDP | Definicion en el proyecto                      |
|----------------|------------------------------------------------|
| Estado (S)     | Categoria del video actual × perfil del agente |

|                |                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| Accion (A)     | ver_completo / ver_parcial / skip / like                                                                          |
| Recompensa (R) | +1 si el siguiente video encaja con el perfil -1 si no encaja +2 si encaja y es mas extremo (mide radicalizacion) |
| Politica (pi)  | epsilon-greedy: explora aleatoriamente al principio, explota lo aprendido con el tiempo                           |

## 5.5 Capa 4 — Big Data y analisis de grafos

Neo4j almacena cada video como un nodo con propiedades (id, titulo, categoria, fecha de captura) y cada recomendacion como una arista dirigida con propiedades (agente, accion, recompensa, timestamp). Sobre este grafo se aplican los siguientes algoritmos:

| Algoritmo              | Libreria  | Que mide                                         |
|------------------------|-----------|--------------------------------------------------|
| PageRank               | Neo4j GDS | Videos mas influyentes / nodos puente            |
| Shortest Path          | Neo4j GDS | Distancia minima a contenido extremo             |
| Louvain Community      | Neo4j GDS | Deteccion automatica de camaras de eco           |
| Betweenness Centrality | Neo4j GDS | Nodos con mayor poder de conexion entre clusters |



## 6. Perfiles de agente

---

Se definen 3 perfiles distintos para maximizar la diversidad de resultados y cubrir diferentes segmentos demograficos relevantes:

### Agente CASUAL

*Perfil demografico: 25 anos, intereses generales*

Entretenimiento neutro, cocina, viajes, humor familiar. Evita contenido politico, violento o muy especializado. Representa al usuario medio de plataformas de video corto.

### Agente GAMER

*Perfil demografico: 18 anos, cultura de internet*

Gaming competitivo, humor absurdo, internet culture, brainrot ironico. Alto umbral de tolerancia a contenido intenso. Representa al segmento mas vulnerable a rabbit holes extremos.

### Agente INFO

*Perfil demografico: 35 anos, perfil informativo*

Noticias, documentales, contenido educativo y divulgativo. Rechaza el entretenimiento vacio. Sirve como grupo de control para comparar con los otros perfiles.

## 7. Funcion de recompensa (reward function)

La funcion de recompensa es el elemento central del Reinforcement Learning y define que comportamiento aprende el agente. En este proyecto se usa una funcion compuesta de tres terminos:

**Definicion formal:**

| R(s, a, s') = R_afinidad + R_extremismo + R_penalizacion |                            |                                                                    |
|----------------------------------------------------------|----------------------------|--------------------------------------------------------------------|
| Termino                                                  | Formula                    | Descripcion                                                        |
| R_afinidad                                               | afinidad(s', perfil) * 1.0 | Recompensa base: el siguiente video encaja con el perfil           |
| R_extremismo                                             | extremismo(s') * 1.0       | Bonus: el video es mas extremo que el anterior (mide polarizacion) |
| R_penalizacion                                           | -1.0 si afinidad < 0.3     | Penalizacion: el siguiente video no encaja con el perfil           |

*Tanto afinidad como extremismo son valores entre 0 y 1 calculados por el propio LLM al analizar el video siguiente. El agente aprende que secuencia de acciones maximiza la recompensa acumulada.*

## 8. Modelo de datos en Neo4j

---

### Esquema de nodos:

```
(:Video { id: STRING, // ID de YouTube titulo: STRING, categoria: STRING, // gaming / politica / educativo / ... extremismo: FLOAT, // 0.0 - 1.0 asignado por el LLM capturado: DATETIME })
```

### Esquema de aristas:

```
(:Video)-[:RECOMIENDA { agente: STRING, // casual / gamer / info accion: STRING, // ver_completo / skip / like recompensa: FLOAT, timestamp: DATETIME }]->(:Video)
```

### Consultas Cypher clave:

#### Camino mas corto desde video neutro a contenido extremo:

```
MATCH (inicio:Video {categoria: 'entretenimiento'}), (fin:Video) WHERE fin.extremismo > 0.8 CALL gds.shortestPath.dijkstra.stream('grafo', { sourceNode: id(inicio), targetNode: id(fin) }) YIELD path RETURN path LIMIT 1
```

#### Deteccion de comunidades (camaras de eco):

```
CALL gds.louvain.stream('grafo') YIELD nodeId, communityId RETURN gds.util.asNode(nodeId).titulo AS video, communityId ORDER BY communityId
```

## 9. Stack tecnologico completo

| Componente      | Tecnologia              | Version | Proposito                           |
|-----------------|-------------------------|---------|-------------------------------------|
| Lenguaje        | Python                  | 3.11+   | Backend orquestador principal       |
| Navegacion      | Playwright              | 1.40+   | Control del navegador Chromium      |
| Transcripciones | youtube-transcript-api  | 0.6+    | Extraccion de subtítulos de YouTube |
| LLM Runtime     | Ollama                  | Latest  | Ejecucion de LLM en local           |
| Modelo LLM      | Llama 3.1 8B            | Q4_K_M  | Motor de decision de los agentes    |
| RL              | NumPy                   | 1.26+   | Q-table y calculo de recompensas    |
| Base de datos   | Neo4j                   | 5.x     | Grafo de recomendaciones            |
| Driver Neo4j    | neo4j (Python)          | 5.x     | Conexion Python-Neo4j               |
| Visualizacion   | neovis.js / Neo4j Bloom | -       | Visualizacion interactiva del grafo |
| Analisis grafos | Neo4j GDS               | 2.x     | PageRank, Shortest Path, Louvain    |
| Utilidades      | pandas, python-dotenv   | -       | Analisis y configuracion            |

### Comandos de instalacion:

```
python -m venv venv && source venv/bin/activate pip install playwright
youtube-transcript-api ollama neo4j numpy pandas python-dotenv playwright install
chromium # Instalar Ollama desde https://ollama.com ollama pull llama3.1 # ~4.5 GB #
Instalar Neo4j Desktop desde https://neo4j.com/download (gratis)
```

## 10. Plan de desarrollo — 7 semanas

| Semana | Fase              | Tareas principales                                                                                      | Entregable                                                                |
|--------|-------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 1      | Setup + Crawler   | Configurar entorno Python. Playwright extrae datos de YouTube Shorts. Guardar datos en JSON local.      | Script funcional que captura 50+ videos y los guarda estructurados        |
| 2      | Agentes LLM       | Definir 3 perfiles. Integrar Ollama. Agente evalua video y devuelve JSON de decision.                   | Agente que toma decisiones coherentes con su perfil sobre cualquier video |
| 3      | Integracion Neo4j | Instalar Neo4j Desktop. Conectar Python. Guardar nodos y aristas durante navegacion.                    | Grafo funcional con 100+ nodos y relaciones tras una sesion de prueba     |
| 4      | Q-Learning        | Implementar Q-table. Definir estados, acciones y funcion de recompensa. Integrar en el bucle principal. | Agente que mejora sus decisiones con el tiempo, metricas de aprendizaje   |
| 5      | Simulacion larga  | Lanzar 3 agentes simultaneos durante 24-48 horas. Monitorizar y depurar.                                | Dataset final: 500-1000+ nodos, 3 subgrafos diferenciados                 |
| 6      | Analisis Big Data | Ejecutar PageRank, Shortest Path y Louvain. Generar graficas y estadisticas.                            | Resultados cuantitativos: distancias, rankings, clusters identificados    |
| 7      | Memoria + Demo    | Redactar memoria tecnica. Preparar visualizacion para defensa. Ensayar presentacion.                    | TFG completo entregable                                                   |

## 11. Minimo producto viable (MVP)

---

Si el tiempo apremia, este es el subconjunto minimo que debe funcionar para obtener una calificacion positiva en todas las asignaturas:

| Componente                                      | Asignatura que cubre              | Imprescindible |
|-------------------------------------------------|-----------------------------------|----------------|
| Playwright extrayendo datos reales de YouTube   | Programacion de IA                | SI             |
| LLM tomando decisiones diferenciadas por perfil | Modelos de IA                     | SI             |
| Q-Learning basico con Q-table                   | Sistemas de Aprendizaje Autonomos | SI             |
| Neo4j con grafo de recomendaciones              | Sistemas Big Data                 | SI             |
| Visualizacion del grafo (Neo4j Bloom)           | Big Data Aplicado                 | SI             |
| Analisis PageRank y Shortest Path               | Big Data Aplicado                 | Recomendado    |
| 3 agentes simultaneos con perfiles distintos    | Todas                             | Recomendado    |
| Simulacion de 24h+ y dataset rico               | Todas                             | Ideal          |

## 12. Resultados esperados

---

### **Grafo de recomendaciones**

500-1000 nodos (videos), 800-2000 aristas (recomendaciones), 3 subgrafos diferenciados por perfil

### **Distancia al contenido extremo**

Medicion en numero de recomendaciones desde un video neutro hasta el primer video con extremismo > 0.8, por perfil

### **Velocidad de atrapamiento**

Numero de videos necesarios hasta que el 80% de las recomendaciones pertenezcan al mismo cluster

### **Nodos puente**

Top 10 videos con mayor Betweenness Centrality: videos que actuan como puerta de entrada a nichos extremos

### **Aislamiento entre perfiles**

Porcentaje de nodos NO compartidos entre el subgrafo del Agente Casual y el Agente Gamer

### **Curva de aprendizaje RL**

Grafica de recompensa acumulada por episodio: demuestra que el agente mejora con el tiempo

## 13. Limitaciones y trabajo futuro

---

### Limitaciones conocidas del sistema:

- Detección de bots por YouTube: se mitiga con delays aleatorios y comportamiento humano simulado, pero no es infalible.
- Sesgo del LLM: Llama 3.1 puede no clasificar perfectamente la categoría de ciertos videos culturalmente específicos.
- Representatividad de perfiles: 3 perfiles no cubren toda la diversidad demografica real.
- Cobertura geografica: el algoritmo de YouTube varia segun region; los resultados son especificos para Espana.
- Ventana temporal: los datos capturados reflejan el estado del algoritmo en el momento de la simulacion.

### Lineas de trabajo futuro:

- Extender el sistema a TikTok con tecnicas de stealth browsing mas avanzadas.
- Aumentar a 10+ perfiles demograficos para mayor representatividad.
- Implementar Deep Q-Network (DQN) para sustituir la Q-table por una red neuronal.
- Anadir analisis de sentimiento a las transcripciones para medir la progresion emocional.
- Publicar el dataset anonimizado como recurso academico abierto.



## 14. Justificacion academica por asignatura

---

### Modelos de IA

Uso de LLM (Llama 3.1 8B) como motor de inferencia semantica. System prompting para asignacion de perfiles psicograficos. Evaluacion de contenido mediante NLU. Generacion de salida estructurada en JSON.

### Programacion de IA

Implementacion del pipeline completo de agentes autonomos en Python. Integracion de Playwright para navegacion controlada por IA. Orquestacion del flujo: extraccion → decision → accion → almacenamiento.

### Sistemas de Aprendizaje Autonomos

Implementacion de Q-Learning como algoritmo de RL. Definicion formal del MDP: estados, acciones, recompensas y politica. Demostracion de aprendizaje mediante curva de recompensa acumulada.

### Sistemas Big Data

Uso de Neo4j como sistema de almacenamiento de datos a escala. Modelado de datos como grafo dirigido con propiedades. Pipeline de ingesta de datos en tiempo real durante la navegacion.

### Big Data Aplicado

Aplicacion de algoritmos de analisis de grafos: PageRank, Shortest Path, Louvain Community Detection, Betweenness Centrality. Visualizacion interactiva del grafo con neovis.js o Neo4j Bloom. Extraccion de conclusiones empiricas sobre polarizacion algoritmica.

## 15. Glosario tecnico

| Termino                       | Definicion                                                                                                                                   |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Agente autonomo               | Sistema de IA que percibe su entorno y toma decisiones de forma independiente sin intervencion humana directa.                               |
| Betweenness Centrality        | Medida de centralidad de un nodo en un grafo: cuantos caminos minimos entre otros nodos pasan por el.                                        |
| Camara de eco                 | Entorno informativo donde el usuario solo recibe contenido que refuerza sus creencias previas.                                               |
| Cypher                        | Lenguaje de consulta declarativo propio de Neo4j para interactuar con bases de datos de grafos.                                              |
| Epsilon-greedy                | Politica de exploracion en RL: con probabilidad epsilon elige una accion aleatoria, el resto del tiempo explota el conocimiento acumulado.   |
| GDS (Graph Data Science)      | Libreria de Neo4j que implementa algoritmos de analisis de grafos como PageRank o Louvain.                                                   |
| Headless browser              | Navegador web que funciona sin interfaz grafica, controlado programaticamente.                                                               |
| LLM (Large Language Model)    | Modelo de lenguaje de gran escala entrenado sobre corpus masivos de texto.                                                                   |
| Louvain                       | Algoritmo de deteccion de comunidades en grafos que agrupa nodos densamente conectados entre si.                                             |
| MDP (Markov Decision Process) | Framework matematico para modelar problemas de decision secuencial con estados, acciones y recompensas.                                      |
| Neo4j                         | Sistema gestor de bases de datos orientado a grafos, lider del sector.                                                                       |
| Nodo puente                   | Video que actua como conector entre dos comunidades o clusters distintos dentro del grafo.                                                   |
| Ollama                        | Herramienta para ejecutar modelos LLM en local sin necesidad de infraestructura cloud.                                                       |
| PageRank                      | Algoritmo de centralidad que mide la importancia de un nodo en funcion de cuantos otros nodos apuntan a el.                                  |
| Playwright                    | Libreria de automatizacion de navegadores web mantenida por Microsoft.                                                                       |
| Q-Learning                    | Algoritmo de Reinforcement Learning que aprende una funcion Q(estado, accion) sin necesidad de conocer el modelo del entorno.                |
| Q-table                       | Tabla que almacena los valores Q aprendidos para cada par (estado, accion).                                                                  |
| Rabbit hole                   | Secuencia de recomendaciones que aleja progresivamente al usuario del contenido inicial hacia nichos cada vez mas especializados o extremos. |
| Reinforcement Learning        | Paradigma de aprendizaje automatico donde un agente aprende mediante prueba y error maximizando una recompensa.                              |

|               |                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------|
| Shortest Path | Algoritmo que encuentra el camino de menor coste entre dos nodos en un grafo.                                         |
| System prompt | Instruccion de contexto que se inyecta al inicio de una conversacion con un LLM para definir su rol o comportamiento. |